TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE CIUDAD MADERO

Materia:
Programación para ciencia de datos

TAREA:
INVESTIGACIÓN NLTK

Alumno:
García Hernández Rodrigo | 23070447
Docente:
Gómez Santillán Claudia Guadalupe
Periodo:
Agosto- diciembre 2026

INVESTIGACIÓN NLTK

LA BIBLIOTECA DEL CONJUNTO DE HERRAMIENTAS DE LENGUAJE NATURAL (NLTK) **3**

ASPECTOS IMPORTANTES DE NLTK **3**

COLECCIÓN DE CORPUS 3
TOKENIZACIÓN Y PROCESAMIENTO DE TEXTO 4
MODELADO DE LENGUAJE 4
ANÁLISIS DE SENTIMIENTOS Y OPINIONES 4
APRENDIZAJES AUTOMÁTICO EN TEXTO 4

TAREAS QUE NLTK PERMITE REALIZAR **4**

TOKENIZACIÓN 4
EJEMPLO DE TOKENIZACIÓN *5*
DERIVACIÓN 5
EJEMPLO DE DERIVACIÓN *5*
ETIQUETADO 6
EJEMPLO DE ETIQUETADO *6*
CLASIFICACIÓN DE TEXTO 6
EJEMPLO BÁSICO DE CLASIFICACIÓN DE TEXTO USANDO NAIVE BAYES EN NLTK *6*
REFERENCIAS BIBLIOGRÁFICAS: 9

LA BIBLIOTECA DEL CONJUNTO DE HERRAMIENTAS DE LENGUAJE NATURAL (NLTK)

Natural Language Toolkit (NLTK) es una popular biblioteca de código abierto para el procesamiento del lenguaje natural (PLN) en Python. Proporciona una interfaz fácil de usar para una amplia gama de tareas, como la tokenización, la separación de palabras, la lematización, el análisis sintáctico y el análisis del sentimiento.

NLTK es ampliamente utilizado por investigadores, desarrolladores y científicos de datos de todo el mundo para desarrollar aplicaciones de PNL y analizar datos de texto.

Una de las principales ventajas de utilizar NLTK es su amplia colección de corpus, que incluye datos de texto de diversas fuentes, como libros, artículos de noticias y plataformas de medios sociales. Estos corpus constituyen una rica fuente de datos para entrenar y probar modelos de PNL.

Se inclyen herramientas para:

- Clasificación
- Tokenización
- Derivación
- Etiquetado
- Análisis sintáctico
- Razonamiento semántico

ASPECTOS IMPORTANTES DE NLTK

COLECCIÓN DE CORPUS

Ofrece una amplia variedad de corpus de texto, que son conjuntos de datos de texto etiquetados y anotados utilizados para entrenar y evaluar modelos de NLP. Estos corpus abarcan una variedad de dominios y lenguajes, lo que permite a los investigadores y desarrolladores acceder a datos de texto de calidad para una amplia gana de aplicación en IA.

TOKENIZACIÓN Y PROCESAMIENTO DE TEXTO

Tiene capacidad de tokenizar texto y realizar el procesamiento de texto, como eliminación de stop words, stemming, lematización y etiquetado gramatical. Son técnicas fundamentales en el procesamiento del lenguaje natural y son utilizadas en una variedad de aplicaciones de IA, como análisis de sentimientos, clasificación de texto y extracción de información.

MODELADO DE LENGUAJE

Proporciona herramientas para construir y entrenar modelos de lenguaje, como modelos de N-Gramas, modelos de markov y modelos de temas. Estos modelos son fundamentales para comprender la estructura y semántica del lenguaje, lo que permite a los sistemas de IA comprender y generar texto de manera más efectiva.

ANÁLISIS DE SENTIMIENTOS Y OPINIONES

Ofrece funcionalidades para el análisis de sentimientos, que es el proceso de determinar la actitud o sentimiento expresado en un texto. Esto es crucial en aplicaciones de IA como la monitorización de redes sociales, análisis de opiniones de clientes y detección de noticias falsas.

APRENDIZAJES AUTOMÁTICO EN TEXTO

Se integra con otras bibliotecas de aprendizaje automático en Python como Scikit-Learn y TensorFlow en Python, lo que permite a los usuarios construir modelos de aprendizaje automático para tareas de NLP. Esto incluye tareas como la clasificación de texto, la generación de texto y el etiquetado de secuencias, entre otras.

TAREAS QUE NLTK PERMITE REALIZAR

TOKENIZACIÓN

Es el proceso de dividir un texto en unidades más pequeñas, llamadas tokens. Estos tokens pueden ser palabras individuales, frases, símbolos de puntuación o incluso caracteres. Es un paso fundamental en muchas tareas de procesamiento de lenguaje natural, ya que proporciona la base para el análisis posterior del texto.

EJEMPLO DE TOKENIZACIÓN

División de una oración en palabras individuales. Usando NLTK, se puede tokenizar una oración en palabras individuales, lo que facilita en análisis posterior del texto.

```
from nltk.tokenize import word_tokenize
nltk.download('punkt')

sentence = "NLTK es una biblioteca de procesamiento de lenguaje natural."
tokens = word_tokenize(sentence)
print(tokens)
```

Imagen 1. Ejemplo de tokenización.

DERIVACIÓN

La derivación en NLTK se refiere al proceso de reducir una palabra a su forma base o raíz. Esto es útil para normalizar el texto y simplificar el análisis, ya que diferentes formas de la misma palabra pueden compartir el mismo significado básico. NLTK proporciona algoritmos de derivación que utilizan reglas lingüísticas o diccionarios para realizar esta tarea, como el algoritmo de Porter Stemmer o el algoritmo de Snowball Stemmer, que pueden aplicarse a diferentes idiomas.

EJEMPLO DE DERIVACIÓN

Reducción de palabras a su forma base.

```
from nltk.stem import PorterStemmer

words = ["running", "plays", "jumped"]
stemmer = PorterStemmer()
stems = [stemmer.stem(word) for word in words]
print(stems)
```

Imagen 2. Ejemplo de derivación

ETIQUETADO

El etiquetado implica asignar etiquetas a las palabras en un texto para indicar su función gramatical o su categoría semántica. Por ejemplo, en el etiquetado gramatical, cada palabra puede ser etiquetada como sustantivo, verbo, adjetivo, etc. En el etiquetado semántico, las palabras pueden ser etiquetadas según su sentido o significado en el contexto específico. NLTK proporciona herramientas para realizar tanto el etiquetado gramatical como semántico utilizando herramientas de aprendizaje automático o reglas lingüísticas.

EJEMPLO DE ETIQUETADO

Etiquetado gramatical de palabras en una oración

```
nltk.download('averaged_perceptron_tagger')
from nltk import pos_tag
from nltk.tokenize import word_tokenize

sentence = "NLTK es una biblioteca de procesamiento de lenguaje natural."
tokens = word_tokenize(sentence)
tagged_words = pos_tag(tokens)
print(tagged_words)
```

Imagen 3. Ejemplo de etiquetado

CLASIFICACIÓN DE TEXTO

Se refiere a la tarea de asignar etiquetas o categorías a los textos en función de ciertos criterios. Por ejemplo, se puede clasificar un sentimiento de un texto como positivo, negativo o neutral en el análisis de sentimientos. NLTK proporciona herramientas y algoritmos para entrenar modelos de clasificación utilizando técnicas de aprendizaje automático, como clasificadores bayesianos, máquinas de soporte vectorial (SVM) o redes neuronales, entre otros.

EJEMPLO BÁSICO DE CLASIFICACIÓN DE TEXTO USANDO NAIVE BAYES EN NLTK

1. IMPORTACIÓN DE BIBLIOTECAS

NLTK: Proporciona herramientas para procesamiento del lenguaje
Natural Random: Genera muestras aleatorias

```
import nltk
import random
```

Imagen 4. Importación de bibliotecas para clasificación

2. DEFINICIÓN DEL CONJUNTO DE DATOS ETIQUETADOS

Data: Lista de tuplas donde cada tupla contiene un texto de comentario de película y su correspondiente etiquetada de sentimiento (positivo o negativo).

```
# Ejemplo de conjunto de datos de textos etiquetados
data = [
    ("I love this movie", "positive"),
    ("This movie is terrible", "negative"),
    ("This movie is great", "positive"),
    ("I dislike this movie", "negative"),
    ("This film is amazing", "positive"),
    ("I can't stand watching this movie", "negative"),
    ("The acting in this movie is phenomenal", "positive"),
    ("I regret wasting my time on this film", "negative"),
    ("I thoroughly enjoyed this movie", "positive"),
    ("This movie lacks depth and substance", "negative"),
    ("The plot of this movie was captivating", "positive"),
    ("I found the characters in this film to be very engaging", "positive"),
    ("The special effects in this movie were impressive", "positive"),
    ("The storyline was predictable and unoriginal", "negative"),
    ("I was disappointed by the lack of character development", "negative"),
    ("The cinematography in this film was stunning", "positive"),
    ("The dialogue felt forced and unnatural", "negative"),
    ("The pacing of the movie was too slow for my liking", "negative"),
    ("I was pleasantly surprised by how much I enjoyed this film", "positive"),
    ("The ending left me feeling unsatisfied and confused", "negative"),
    ("This movie exceeded my expectations", "positive"),
    ("The performances by the actors were lackluster", "negative")
]
```

Imagen 5. Conjunto de datos etiquetados

3. PREPROCESAMIENTO DE DATOS

Función que toma un texto como entrada y realiza el procesamiento básico, que en este caso es la tokenización. Cada token (palabra) en el texto se convierte en una característica con un valor booleano verdadero (True)

```
# Preprocesamiento de datos: tokenización y extracción de características
def preprocess(text):
    tokens = nltk.word_tokenize(text)
    return {word: True for word in tokens}
```

Imagen 6. Preprocesamiento de datos

4. APLICACIÓN DE PREPROCESAMIENTO A LOS DATOS

Featuresets: Lista de tuplas donde cada tupla contiene un diccionario de características preprocesadas y su correspondiente etiqueta. Las características se extraen utilizando la función preprocess() y se almacenan como diccionarios.

```
# Aplicamos el preprocesamiento a los datos
featuresets = [(preprocess(text), label) for (text, label) in data]
```

Imagen 7. Aplicación del preprocesamiento

5. DIVISIÓN DE DATOS

Train_set, test_set: Se dividen los datos preprocesados en conjuntos de entrenamiento y prueba. Aquí, los primeros 16 elementos se utilizan para el conjunto de entrenamiento y el resto para el conjunto de prueba.

```
# Dividimos los datos en conjuntos de entrenamiento y prueba
train_set, test_set = featuresets[:16], featuresets[16:]
```

Imagen 8. Asignación de datos para conjunto de entrenamiento y para pruebas

6. ENTRENAMIENTO DEL CLASIFICADOR

Nltk.NaiveBayesClassifer.Train(train_set): Se entrena un clasificador Naive Bayes utilizando el conjunto de entrenamiento

```
# Entrenamos un clasificador utilizando Naive Bayes
classifier = nltk.NaiveBayesClassifier.train(train_set)
```

Imagen 9. Asignamos conjunto de entrenamiento para Naive Bayes

7. EVALUACIÓN DEL CLASIFICADOR

Nltk.classify.accuracy(classifier, test_set): Se evalúa la precisión del clasificador en el conjunto de prueba y se imprime el resultado.

```
# Evaluamos el clasificador en el conjunto de prueba
accuracy = nltk.classify.accuracy(classifier, test_set)
print("Accuracy:", accuracy)
```

Imagen 10. Evaluación del clasificador

8. CLASIFICACIÓN DE UN NUEVO TEXTO

Se proporciona un nuevo texto (new_text) que se desea clasificar. Se procesa el nuevo texto (preprocess(new_text)) para convertirlo en caracteristicas. Se utiliza el clasificador entrenado para predecir la etiqueta del nuevo texto y se imprime el resultado.

```
# Clasificamos un nuevo texto
new_text = "This movie is amazing"
new_text_features = preprocess(new_text)
predicted_label = classifier.classify(new_text_features)
print("Predicted label:", predicted_label)
```

Imagen 11. Clasificación del texto nuevo

REFERENCIAS BIBLIOGRÁFICAS:

- https://www.nltk.org
- https://sites.google.com/view/programacion-en-python/home/3-natural-language-toolkit
- https://imaster.academy/contenidos-tematicos/talentotech/TalentoTech/InteligenciaArtificial2/Mision2/Unidad3/6.-IntroduccionNLTKNaturalLanguageToolkit.pdf.pdf
- https://www.datacamp.com/es/tutorial/text-analytics-beginners-nltk